Worst Case Analysis

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **classify** a loop that halves or doubles as O(log N).
2. Be able to **identify** the best and worst case of a search.
3. Be able to **distinguish** O, Ω, and Θ.

Recap: add or multiply?

- One loop after another: **add**. N + N is O(N).
- One loop inside another: **multiply**. N × N is O(N²).
- A method call can hide a loop: `countMissing` was M road-trip songs × N library songs, **O(M × N)**.

What's the Big-O?

```
int total = n * (n + 1) / 2;                // O(1)
for (int i = 0; i < 5; i++) { ... }         // O(1): always 5 passes
String title = library[500];                // O(1)
char c = title.charAt(3);                   // O(1)
boolean b = title.equals("Hey Jude");       // O(1)
```

Same steps, however big the library gets: **O(1)**.

`equals` takes longer for a longer title, but titles don't grow with the library.

What's the Big-O?

```
for (int i = 0; i < library.length; i++) { ... }     // O(N)
for (String title : library) { ... }                 // O(N)
contains(library, "Mr. Brightside");                 // O(N): a loop inside
```

One pass over the library: **O(N)**.

What's the Big-O?

```java
for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) { ... }   // O(N²)
}

for (int i = 0; i < n; i++) {
  for (int j = i; j < n; j++) { ... }   // still O(N²)?
}
```

Yes! N + (N - 1) + ... + 1 is about half of N², still **O(N²)**.

Like Monday's `hasDuplicateTitles`.

What's the Big-O?

```
for (int i = 1; i < n; i *= 2) { ... }   // 1, 2, 4, 8, 16, ...
```

For n = 1,000, that's 10 passes. For n = 2,000, 11.

Doubling (or halving) each pass: **O(log N)**.

log N is how many doublings get you from 1 to N, or halvings from N to 1. For 1,000, about 10.

Eclipse Exercise

Get today's files from Canvas, and open `BinarySearch.java`.

Run it. Double the songs, and the steps...?

Double the songs, one more step

Songs	`binarySearch` steps
1,000	9
2,000	10
4,000	11

Each pass throws out half the titles. A million songs? About **20 steps**.

Tophat Q1

What's the Big-O?

```
for (int i = n; i > 0; i = i / 2) {
  steps++;
}
```

→ Open Tophat to vote.

Tophat Q2

What's the Big-O?

```
for (int i = 1; i < n; i *= 2) {
  for (int j = 0; j < n; j++) {
    steps++;
  }
}
```

→ Open Tophat to vote.

Worst case analysis

1. What input takes the **most** steps? How many?
2. What input takes the **fewest** steps? How many?
3. Are they the **same**?

Eclipse Exercise

Open `IndexOf.java`. `main` searches for the first song, the last one, and one that isn't there.

TODO 1. Guess the steps for each. Which is the best case? The worst? Then run it.

`indexOf` : best and worst

Search for...	Steps
the first song	1
the last song	1,000
a missing song	1,000

Best case: **1 step**. Worst case: **N steps**. Not the same!

O, Ω, and Θ

	Means	`indexOf`	`totalSeconds`
O	at most: the worst case	O(N)	O(N)
Ω	at least: the best case	Ω(1)	Ω(N)
Θ	both are the same	no Θ	Θ(N)

Ω is "big omega". Θ is "big theta".

Binary search: best and worst

- Best case: the title is in the middle. **1 step: Ω(1).**
- Worst case: the title isn't there. **About log N steps: O(log N).**

Not the same, so **no Θ**.

Tophat Q3

What are `hasDuplicateTitles`'s best and worst cases?

```
for (int i = 0; i < titles.length; i++) {
  for (int j = i + 1; j < titles.length; j++) {
    if (titles[i].equals(titles[j])) {
      return true;
    }
  }
}
return false;
```

→ Open Tophat to vote.

Tophat Q4

What are `findMax`'s best and worst cases?

```
int max = seconds[0];
for (int i = 1; i < seconds.length; i++) {
  if (seconds[i] > max) {
    max = seconds[i];
  }
}
```

→ Open Tophat to vote.

Questions?